Домашнее задание

Deformable DETR: разбор архитектуры и запуск pretrained-модели

Ruslan Papina, группа M25-555

1. Статья

Выбрал Deformable DETR. От неё идут почти все следующие DETR-подобные детекторы (DAB-DETR, DN-DETR, DINO), а её главная идея хорошо проверяется на экспериментах из самой статьи.

Название	Deformable DETR: Deformable Transformers for End-to-End Object Detection
Авторы	Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, Jifeng Dai (SenseTime Research, USTC, CUHK)
Год, конференция	ICLR 2021, oral. Препринт на arXiv - октябрь 2020
Статья	https://arxiv.org/abs/2010.04159 https://openreview.net/forum?id=gZ9hCDWe6ke
Официальный код	https://github.com/fundamentalvision/Deformable-DETR (PyTorch, CUDA-ядро для deformable attention)
Веса, которые запускал	https://huggingface.co/SenseTime/deformable-detr (конвертированы из официального репозитория)

2. Место в развитии детекции

Faster R-CNN + FPN → DETR → Deformable DETR → DAB-DETR / DN-DETR → DINO

год	модель	что дала и чем мешала
2015-2017	Faster R-CNN + FPN	Двухстадийный детектор: якоря, RPN, NMS. FPN строит пирамиду признаков, мелкие объекты ищутся на карте stride 8. Много ручных компонентов.
2017	Deformable Convolution (DCN)	Свёртка берёт значения не в фиксированной сетке, а в точках со смещениями, которые предсказывает сама сеть. Общий автор с Deformable DETR (Jifeng Dai).
2020	DETR (ECCV)	CNN + трансформер-энкодер-декодер, 100 object queries, венгерское сопоставление один-к-одному. Нет якорей и NMS. Учится 500 эпох и плохо видит мелкие объекты.
2021	Deformable DETR (ICLR)	Схема DETR (запросы, сопоставление, лосс) + идея DCN: внимание смотрит в несколько обучаемых точек вокруг опорной, сразу на 4 масштабах.
2022-2023	DAB-DETR, DN-DETR, DINO	Запросы как явные якорные боксы, denoising-запросы, выбор запросов из энкодера как в two-stage. У DAB-DETR и DN-DETR есть deformable-версии, DINO целиком на deformable attention.

3. Какую проблему решают авторы

У DETR два конкретных ограничения, и у обоих одна причина - плотное внимание по всем пикселям карты признаков.

Медленная сходимость. DETR на COCO учится 500 эпох, это около 2000 GPU-часов V100 (больше 10 суток на 8 GPU), DETR-DC5 - 7000 часов. Faster R-CNN + FPN доходит до того же AP за 109 эпох и 380 часов. Если DETR-DC5 обучить всего 50 эпох, получается 35.3 AP вместо 43.3. Авторы объясняют это так: при инициализации веса внимания почти равномерные, примерно $1/N_k$ на каждый из $N_k = H \cdot W$ пикселей. Градиент размазан по всей карте, и сети нужны сотни эпох, чтобы научиться смотреть в несколько точек на краях объекта.

Мелкие объекты. AP_S у DETR 20.5, у Faster R-CNN + FPN 26.6. Обычные детекторы ищут мелкие объекты на карте stride 8, а DETR работает только на C5 (stride 32), потому что self-attention в энкодере стоит $O(H^2W^2C)$. Для картинки 800×1066 C5 - это 25×34 = 850 токенов, а карта stride 8 - 13 400 токенов, пар для внимания в 250 раз больше. Уже DC5 (stride 16) даёт 187 GFLOPs и 12 FPS.

Почему это важно: по данным COCO около 41% объектов в датасете маленькие (меньше 32×32), так что слабый AP_S бьёт по общей метрике. А 500 эпох обучения означают, что любой эксперимент с архитектурой занимает недели, и DETR неудобно развивать дальше.

4. Архитектура

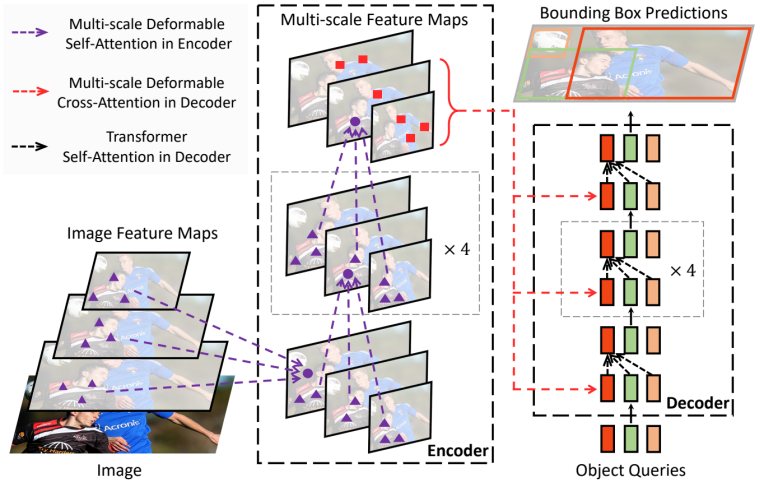


Рис. 1. Схема Deformable DETR из статьи (Figure 1)

Путь данных от картинки до предсказания:

1. Backbone. ResNet-50, берутся выходы C3, C4, C5 (stride 8, 16, 32). Каждый проходит conv 1×1 до 256 каналов, четвёртый уровень получается conv 3×3 со stride 2 из C5 (stride 64). FPN нет. Для 800×1066 это 100×134, 50×67, 25×34 и 13×17, всего около 17 800 токенов.

2. Кодирование позиции. К каждому токену прибавляется синусное позиционное кодирование и обучаемый эмбеддинг уровня e_l, чтобы энкодер различал масштабы.

3. Энкодер, 6 слоёв. Каждый токен - запрос, его опорная точка - он сам. В multi-scale deformable self-attention линейный слой из признака запроса z_q предсказывает смещения Δp для каждой головы (M = 8), уровня (L = 4) и точки (K = 4), второй линейный слой с softmax даёт веса A. Значения берутся билинейной интерполяцией в точках $p_q + \Delta p$ на всех уровнях и суммируются с весами (рис. 2):

$$\mathrm{MSDeformAttn}(z_q, p_q, \{x_l\}) = \Sigma_m W_m \left[\Sigma_l \Sigma_k A_{mlqk} \cdot W'_m x_l(\varphi_l(p_q) + \Delta p_{mlqk}) \right]$$

Каждый токен смотрит в 8·4·4 = 128 точек вместо всех 17 800, сложность энкодера становится линейной по H·W.

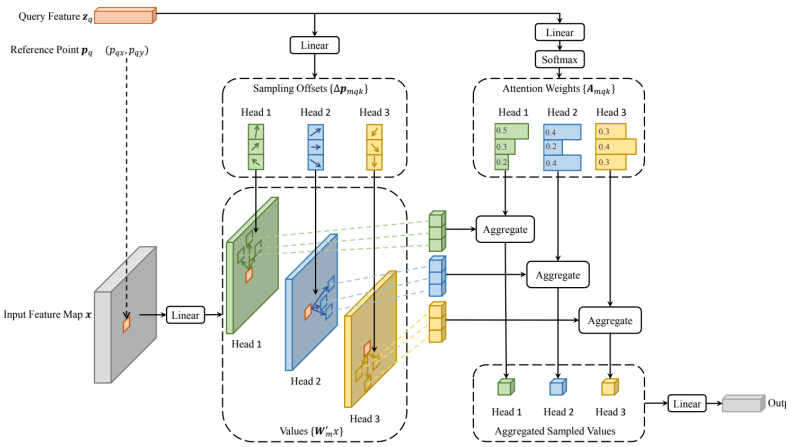


Рис. 2. Модуль deformable attention из статьи (Figure 2): из запроса получаются смещения и веса, значения сэмплируются в смещённых точках

4. Декодер, 6 слоёв, 300 object queries. Self-attention между запросами обычный, как в DETR: 300 запросов - это немного. Cross-attention заменён на multi-scale deformable: опорная точка запроса получается линейным слоем и сигмоидой из его позиционного эмбеддинга, и запрос

сэмплирует точки вокруг неё на выходах энкодера.

5. Головы. Линейный классификатор на 91 класс с сигмоидой (focal loss) и MLP из 3 слоёв для бокса. Бокс предсказывается как поправка к опорной точке, она служит начальной оценкой центра.

6. Обучение и выход. Как в DETR: 300 предсказаний сопоставляются с разметкой венгерским алгоритмом один-к-одному, лосс focal + L1 + GIoU. Поэтому NMS нет. На выходе берутся топ-100 пар (запрос, класс) по скору и отсекаются порогом.

5. Чем отличается от DETR

компонент	DETR	Deformable DETR
признаки	только C5, stride 32	C3-C6, stride 8-64, 4 уровня, без FPN
self-attention в энкодере	плотное, $O((HW)^2C)$	multi-scale deformable, 4 точки на голову и уровень, $O(HWC^2)$
cross-attention в декодере	плотное по всей карте	multi-scale deformable вокруг опорной точки
self-attention в декодере	обычное	без изменений
бокс	абсолютные координаты	поправка к опорной точке
запросы / лосс классов	100 / softmax + CE	300 / сигмоида + focal loss
эпох обучения	500	50

Дополнительно в статье есть два варианта: iterative bounding box refinement (каждый слой декодера уточняет бокс предыдущего) и two-stage (энкодер сам предлагает кандидатов, они становятся запросами декодера).

Почему это должно помогать. Во-первых, у запроса всего несколько ключей, и на старте обучения внимание не размазано по 17 тысячам пикселей, а сосредоточено рядом с опорной точкой. Градиент идёт в конкретные смещения, искать объект по всей карте сотни эпох не нужно. Во-вторых, сложность линейная, поэтому можно взять карту stride 8, на которой мелкие объекты занимают больше одной клетки. В-третьих, точки берутся сразу на всех уровнях, так что внимание само смешивает масштабы и FPN становится не нужен. И последнее: бокс привязан к опорной точке, вокруг которой смотрит внимание, поэтому то, куда смотрит декодер, и то, что он предсказывает, связаны напрямую. Focal loss и 300 запросов - не основное изменение, авторы их отдельно контролируют через DETR-DC5+.

6. Эксперименты авторов

Датасет - COCO 2017: train (118 тыс. картинок) для обучения, val (5 тыс.) для сравнения с DETR и ablation, test-dev для сравнения с другими детекторами. Метрики: COCO AP (среднее по IoU от 0.5 до 0.95), AP50, AP75 и AP по размеру объекта (S меньше 32^2, M до 96^2, L больше), плюс параметры, FLOPs, GPU-часы обучения и FPS на V100. Сравнивают с Faster R-CNN + FPN, DETR, DETR-DC5 и DETR-DC5+ (табл. 1), а в табл. 3 с FCOS, ATSS, TSD и EfficientDet-D7.

Таблица 1. Сравнение с DETR на COCO val (табл. 1 статьи)

метод	эпох	AP	AP_S	AP_M	AP_L	FLOPs	GPU-часы	FPS
Faster R-CNN + FPN	109	42.0	26.6	45.4	53.4	180G	380	26
DETR	500	42.0	20.5	45.8	61.1	86G	2000	28
DETR-DC5	500	43.3	22.5	47.3	61.1	187G	7000	12
DETR-DC5	50	35.3	15.2	37.5	53.6	187G	700	12
DETR-DC5+ (focal loss, 300 запросов)	50	36.2	16.3	39.2	53.9	187G	700	12
Deformable DETR	50	**43.8**	**26.4**	47.1	58.0	173G	325	19
+ iterative box refinement	50	45.4	26.8	48.3	61.7	173G	325	19
++ two-stage	50	46.2	28.8	49.2	61.7	173G	340	19

Как я её читаю. Главная строка для сравнения - DETR-DC5+: те же 50 эпох, тот же focal loss и 300 запросов. У него 36.2 AP, у Deformable DETR 43.8, по мелким объектам 16.3 против 26.4. Значит, выигрыш не из-за лосса и числа запросов. Против полного DETR на 500 эпохах: +1.8 AP при в 10 раз меньшем числе эпох и в 6 раз меньшем числе GPU-часов, а AP_S вырос с 20.5 до

26.4 и почти догнал Faster R-CNN + FPN. При этом AP_L упал с 61.1 до 58.0: большим объектам глобальное внимание DETR помогает, а нескольких точек вокруг опорной может не хватить на весь объект. С iterative refinement AP_L возвращается (61.7). По FLOPs модель на уровне DETR-DC5, но работает 19 FPS против 12, потому что плотное внимание упирается в доступ к памяти. Обычного DETR (28 FPS) она всё равно медленнее.

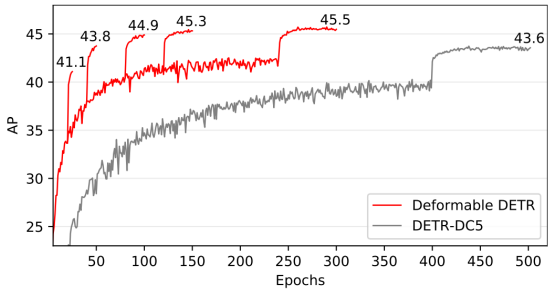


Рис. 3. Кривые сходимости из статьи (Figure 3): Deformable DETR выходит на 43.8 AP к 50 эпохам, DETR-DC5 к этому уровню идёт 500 эпох

В табл. 3 с ResNeXt-101 + DCN и test-time аугментациями получается 52.3 AP на test-dev, на уровне EfficientDet-D7 (52.2) и выше ATSS (50.7) и TSD (51.2). В README репозитория у выложенной модели 44.5 AP вместо 43.8: авторы пишут, что разница из-за переноса кода с внутренней кодовой базы. Эти веса я и запускал.

7. Ablation study

Таблица 2. Ablation deformable attention на COCO val (табл. 2 статьи)

MS inputs	MS attention	K	FPN	AP	AP_S	AP_M	AP_L
✓	✓	4	FPN	43.8	26.5	47.3	58.1
✓	✓	4	BiFPN	43.9	25.6	47.4	57.7
		1	нет	39.7	21.2	44.3	56.0
✓		1	нет	41.4	24.1	44.6	56.1
✓		4	нет	42.3	24.8	45.1	56.3
✓	✓	4	нет	43.8	26.4	47.1	58.0

MS inputs - подавать четыре уровня признаков вместо одного, MS attention - может ли запрос брать точки на всех уровнях сразу или только на своём, K - число точек на голову и уровень.

Разберу переход между двумя последними строками без FPN. Многоуровневые входы и K = 4 уже есть, отключается только межуровневое внимание: каждый токен сэмплирует точки лишь на своей карте, и масштабы между собой не обмениваются. AP падает с 43.8 до 42.3, AP_S с 26.4 до 24.8, AP_L с 58.0 до 56.3. Если же межуровневое внимание оставить и сверху добавить FPN или BiFPN, ничего не меняется (43.8 и 43.9). Вывод: multi-scale deformable attention само делает работу FPN, отдельная пирамида не нужна. Остальные шаги: многоуровневые входы дают +1.7 AP и +2.9 AP_S, K = 4 вместо 1 ещё +0.9.

Ещё один полезный факт из README: single-scale вариант (только res5, stride 32, как у DETR) за 50 эпох даёт 39.4 AP и 20.6 AP_S. Это больше, чем у DETR-DC5+ (36.2) с более подробными признаками stride 16 и тем же числом эпох. То есть разреженная выборка ускоряет обучение и без многоуровневости.

8. Запуск pretrained-модели

Ноутбук detection.ipynb. Модель SenseTime/deformable-detr через transformers 5.22, для сравнения SenseTime/deformable-detr-single-scale (README: 39.4 AP) и facebook/detr-resnet-50 (42.0 AP). Всё считалось на CPU Apple M2, процессор модели ресайзит картинку до 800 по короткой стороне. Взял 15 картинок: 11 из COCO val2017 (на ней модель не обучалась) и 4 с Wikimedia Commons, уменьшенные до 640 по длинной стороне, как в COCO.

картинки	откуда	что проверяю
01 skier, 02 giraffes, 03 fruits, 04 stop sign	COCO val	простые случаи, перекрытие, похожие классы

картинки	откуда	что проверяю
05 cat tv, 06 kids, 07 class photo	COCO val	объект на экране, группа людей, плотная ч/б толпа
08 living room, 09 kitchen, 10 airplanes, 11 train	COCO val	много мелких объектов, далёкие объекты
12 kangaroos, 13 lynx	Wikimedia	животные, которых нет среди классов COCO
14 bruegel, 15 parking aerial	Wikimedia	картина Брейгеля, парковка с дрона

Скоры у Deformable DETR из-за сигмоиды и focal loss заметно ниже, чем у DETR с softmax. При пороге 0.5 на классной фотографии у неё остаются только галстуки и ни одного человека. Поэтому порог свой: 0.3 для Deformable DETR, 0.7 для DETR. При них на 15 картинок получается сопоставимое число боксов (153 и 169, график в ноутбуке).

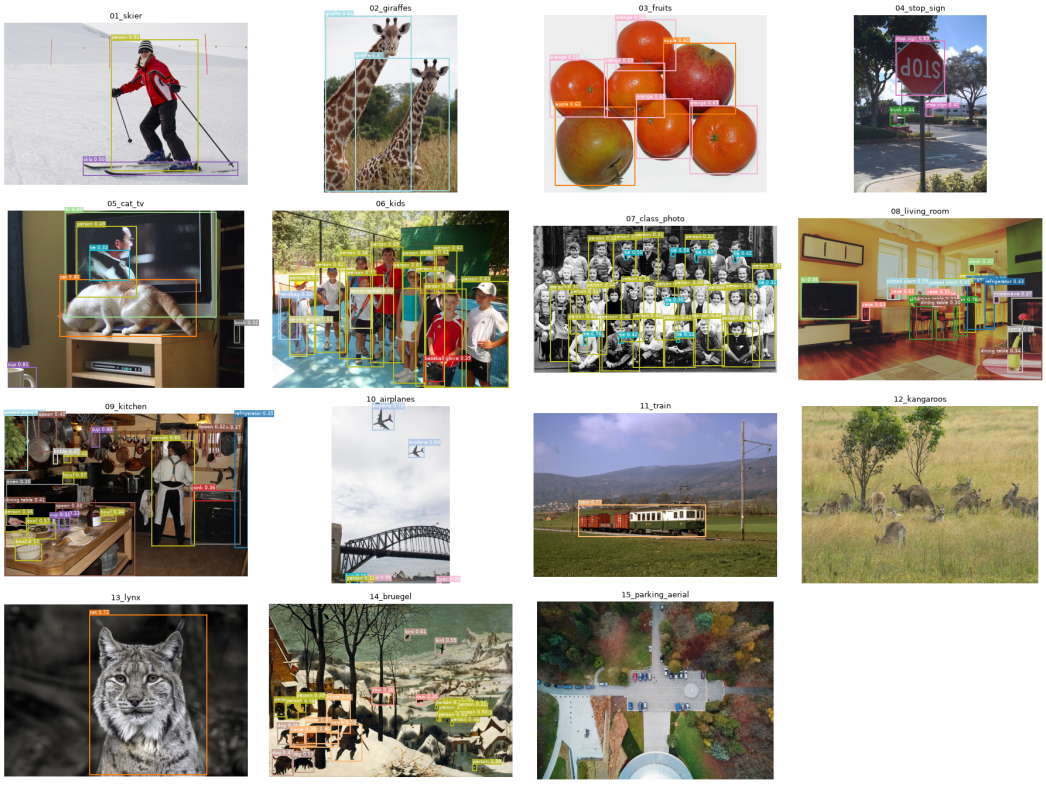


Рис. 4. Deformable DETR на всех 15 картинках, порог 0.3

На простых картинках (лыжник, жирафы, фрукты, рысь, поезд) всё хорошо, даже перевёрнутый знак STOP найден с 0.89. Характерные ошибки на рис. 5:

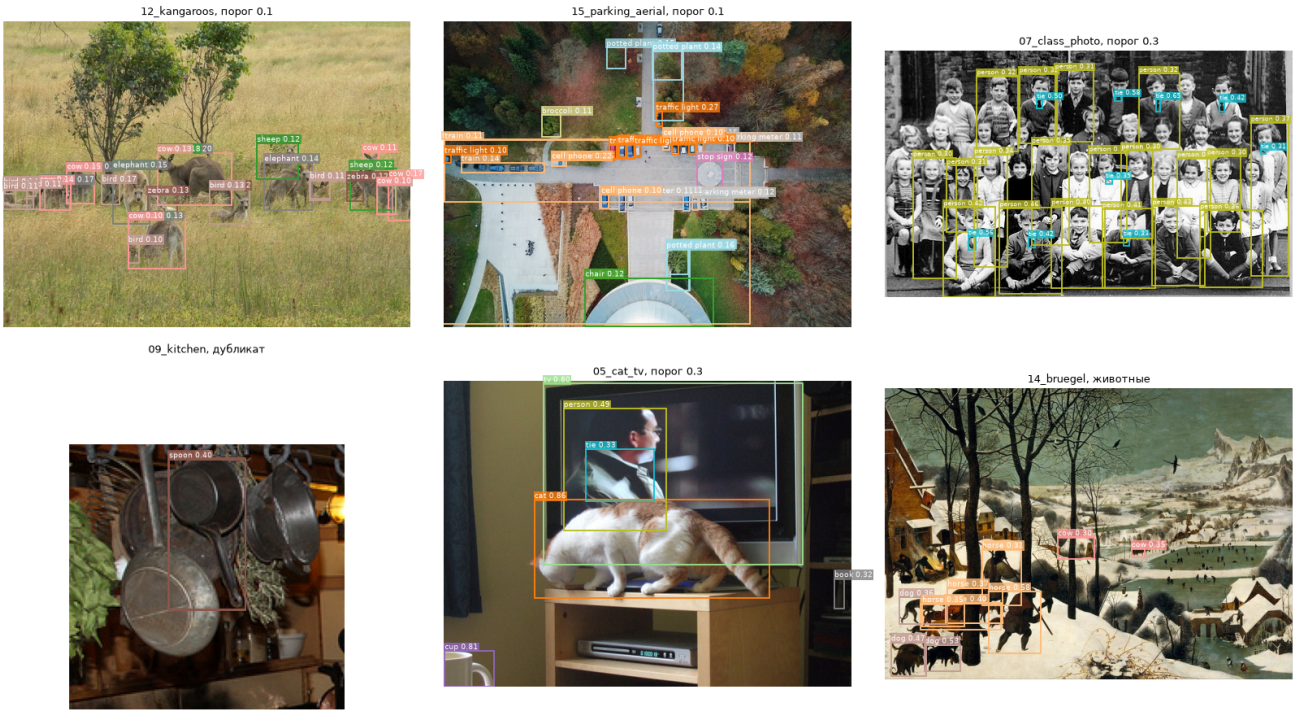


Рис. 5. Ошибки: кенгуру и парковка при пороге 0.1, толпа, дубликат, человек на экране, животные на картине

1. Класс не из COCO (12_kangaroos). При пороге 0.3 не найдено ничего, лучшие ответы sheep 0.26, bird 0.23, zebra 0.22. У модели 91 выход с сигмоидой на каждый класс и нет класса для неизвестного объекта. Кенгуру немного похож на несколько животных сразу, и ни один класс не набирает уверенности. DETR на том же кадре уверенно говорит sheep 0.85, то есть это ограничение данных (фиксированные 80 классов), а не deformable attention. Рысь, наоборот, уверенно стала cat 0.72. Эту проблему решают уже детекторы с открытым словарём (Grounding DINO, YOLO-World).

2. Необычный ракурс (15_parking_aerial). Около 20 машин сверху, каждая примерно 15×30 пикселей. Лучший ответ car 0.28, и тот на куполе планетария внизу кадра. На рядах машин только traffic light, cell phone и parking meter с 0.1-0.27. В COCO машины сняты с уровня человека, а сверху машина - прямоугольник со стеклом, правда похожий на телефон. Даже на карте stride 8 такая машина занимает 2×4 клетки. DETR ошибается увереннее: clock 0.95 на круглой площадке и toilet 0.67 на куполе, а лучшая настоящая машина у левого края у него car 0.6, ниже порога. Многоуровневые признаки не помогают, если такого вида объекта не было в обучении.

3. Плотная толпа (07_class_photo). Около 40 детей на кадре 500×302, лица по 20 пикселей, фото чёрно-белое. Deformable DETR даёт 19 person со скорами 0.3-0.46, часть детей пропущена, некоторые боксы захватывают двоих, а галстуки находятся увереннее людей. DETR при 0.7 находит 25 человек с более точными границами. Моё объяснение: запрос смотрит всего в 4 точки на голову и уровень вокруг опорной, и когда рядом стоят одинаковые объекты, точки легко попадают на соседа, бокс растягивается. Плотное cross-attention DETR видит всю карту и разделяет соседей лучше. Плюс в COCO почти нет ч/б фото.

4. Дубликаты без NMS (09_kitchen и др.). Венгерское сопоставление учит отдавать каждый объект одному запросу, а гасить копии должен self-attention в декодере. При низком пороге копии проходят: spoon 0.45 и 0.40 с IoU 0.97 (причём это висящая сковорода с длинной ручкой, а не ложка), vase 0.32 и 0.31, handbag 0.40 и 0.33, person 0.57 и 0.50 на картине. Порог приходится держать низким из-за калибровки скоров, поэтому дубликатов больше, чем у DETR при 0.7 (у него два, оба на картине). На практике тут помог бы обычный NMS поверх выхода.

5. Картина (14_bruegel). Собаки охотников стали horse (6 боксов, 0.31-0.58), на льду две cow, мелкие фигурки конькобежцев находятся через раз. Это сдвиг домена: мазки и

пропорции не как на фото, а фигурки по 10 пикселей. Спорный случай - человек на экране телевизора (05_cat_tv): person 0.49 и tie 0.33. Модель не отличает изображённого человека от настоящего, но в COCO такие люди тоже бывают размечены.

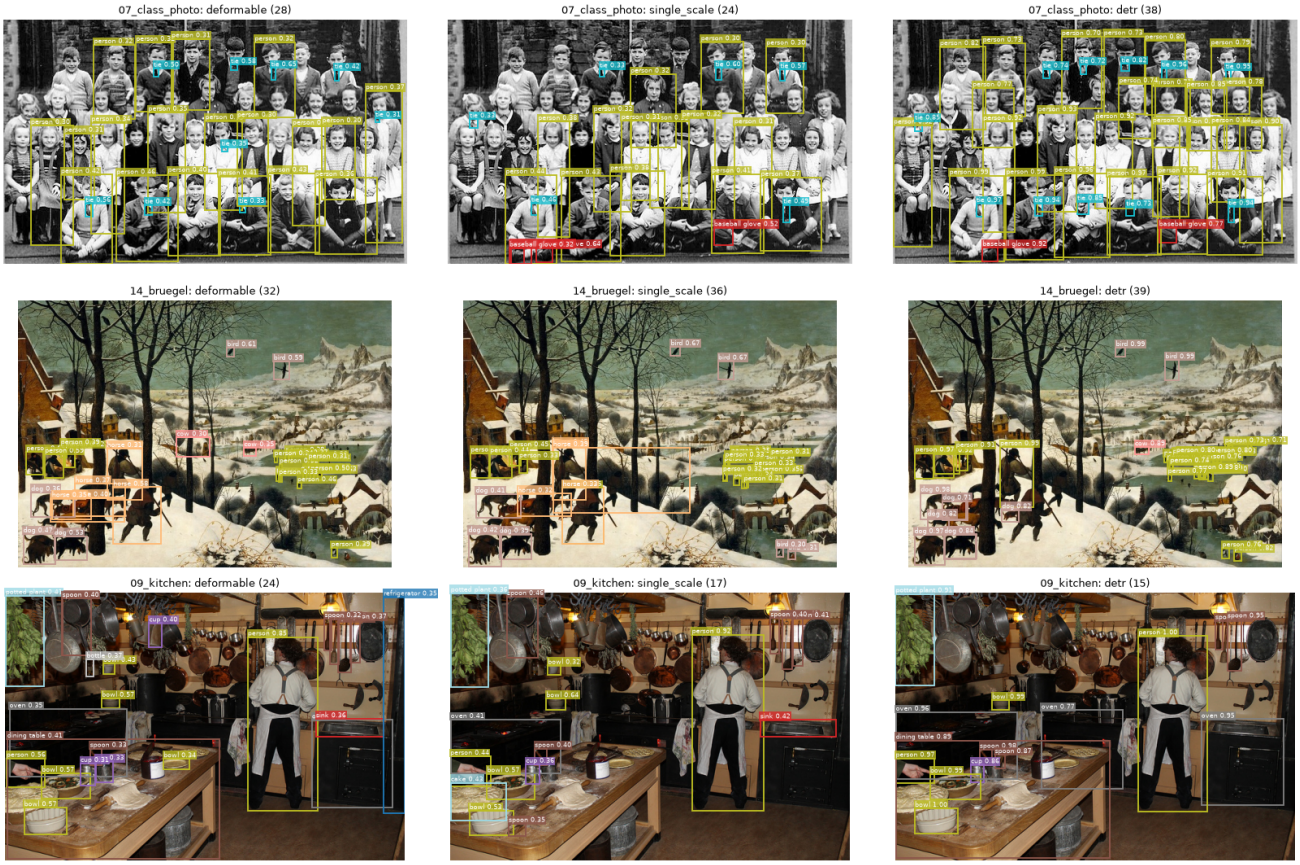


Рис. 6. Одни и те же картинки: Deformable DETR (0.3), single-scale (0.3), DETR (0.7). В скобках число боксов

Сравнение моделей. Всего боксов 153 / 156 / 169 (deformable / single-scale / DETR), мелких меньше 32×32 - 56 / 56 / 67. На кухне multi-scale модель находит больше мелкого (10 против 6 и 5), но там и правильные миски с чашками, и ложные spoon на шумовке и sink на плите. Разметки у меня нет, поэтому AP_S на своих картинках не посчитать. По времени на CPU: 4.0 с на картинку у Deformable DETR, 1.1 с у DETR и 1.0 с у single-scale. На CPU deformable attention считается через grid_sample в PyTorch без CUDA-ядра, а уровень stride 8 даёт в 16 раз больше токенов, чем C5.

9. Выводы

Главное отличие Deformable DETR от DETR - плотное внимание по картам признаков (self-attention в энкодере и cross-attention в декодере) заменено на multi-scale deformable attention: запрос берёт по 4 точки на голову и уровень вокруг своей опорной точки, смещения и веса предсказываются из самого запроса. Одно это изменение убирает квадратичную сложность, а с ней запрет на карты stride 8, и убирает равномерное внимание на старте обучения.

Доказательства в статье есть, но с оговорками. Табл. 1: против DETR-DC5+ с тем же числом эпох, focal loss и 300 запросами +7.6 AP и +10.1 AP_S, так что дело не в лоссе и не в числе запросов. Но вместе с вниманием поменялись входы (4 уровня вместо одного) и параметризация бокса через опорную точку, так что это вклад всего пакета. Табл. 2 разбирает части внутри deformable-семейства: многоуровневые входы, K и межуровневое внимание дают +1.7, +0.9 и +1.5 AP, FPN сверху не нужен. Самое чистое сравнение для сходимости - single-scale вариант из README: признаки res5, как у DETR, 50 эпох, 39.4 AP против 36.2 у DETR-DC5+. Эксперимента, где плотное внимание работает на тех же четырёх уровнях, нет: по памяти это и невозможно, в этом весь смысл. Отдельной проверки опорных точек без deformable-выборки тоже нет. Поэтому я бы сказал так: польза основного

изменения подтверждена, но полностью от остальных изменений не отделена.

Мои картинки это не подтверждают и не опровергают: без разметки AP_S не посчитать, а по числу мелких боксов multi-scale и single-scale модели вышли одинаково. Зато видно, что большинство ошибок связано с данными (класс не из COCO, вид сверху, картина, ч/б толпа), а не с вниманием, и что скоры Deformable DETR надо калибровать отдельно: с порогом от DETR она теряет большую часть объектов.

Источники

1. Zhu et al. Deformable DETR, ICLR 2021. https://arxiv.org/abs/2010.04159, https://openreview.net/forum?id=gZ9hCDWe6ke
2. Официальный код и README с результатами: https://github.com/fundamentalvision/Deformable-DETR
3. Carion et al. End-to-End Object Detection with Transformers (DETR), ECCV 2020. https://arxiv.org/abs/2005.12872
4. Dai et al. Deformable Convolutional Networks, ICCV 2017. https://arxiv.org/abs/1703.06211
5. Веса: https://huggingface.co/SenseTime/deformable-detr, https://huggingface.co/facebook/detr-resnet-50
6. Статистика размеров объектов COCO: https://cocodataset.org/#detection-eval